

Notes on Neural Scorers and Sequence Encoders

Supplementary material to Paper 5, *Sequence encoding of run histories*

Patrick Gillen

2026-09-09

Table of contents

Purpose	2
1 Neural networks	2
1.1 A unit	2
1.2 Layers	2
1.3 Universal approximation, stated honestly	3
2 Training	3
2.1 The loss and gradient descent	3
2.2 Backpropagation	3
2.3 Optimisers, epochs, minibatches	4
2.4 Overfitting, and why a validation slice exists	4
3 The Plackett–Luce likelihood under a neural scorer	4
3.1 The likelihood	5
3.2 What the gradient hands to the network	6
3.3 Race as the grouping unit	6
3.4 Shared weights and variable field sizes	6
4 What an MLP can represent that a linear score cannot	7
4.1 The linear case	7
4.2 Interactions must be named	7
4.3 Thresholds and saturation	7
4.4 Race-level features	8
4.5 What the network does not buy	8
5 Embeddings	8
5.1 The construction	8
5.2 Why an embedding can carry more than a rate	9
5.3 The sample-size problem	9
5.4 The information-set point	9

6	Sequence encoders	10
6.1	The problem	10
6.2	Recurrence	10
6.3	The gated recurrent unit	10

Purpose

These notes cover the machinery paper 5 uses. They are a tutorial, not a report: there are no results here, and nothing is claimed as new. The aim is that a reader with undergraduate mathematics can follow what paper 5 fitted and why the pieces are arranged as they are, or fill in the details independently from here.

The order is cumulative. Each section uses only what precedes it. Sections 1 and 2 are standard and kept short; sections 3 and 4 are where general machinery meets this particular likelihood, and are the point of the note.

Notation: $x \in \mathbb{R}^p$ is a feature vector for one runner in one race, θ is the collection of all model parameters, and $\sigma(u) = (1 + e^{-u})^{-1}$.

1 Neural networks

1.1 A unit

A single unit takes an input vector $x \in \mathbb{R}^p$, forms a weighted sum, and applies a fixed nonlinear function:

$$a = \varphi(w^\top x + b), \quad w \in \mathbb{R}^p, b \in \mathbb{R}.$$

The weights w and the bias b are learned; φ is chosen in advance. The common choice, and the one paper 5 uses, is the rectified linear unit $\varphi(u) = \max(0, u)$.

1.2 Layers

A layer applies m units to the same input. Each unit has its own weights and bias, so the m weighted sums are the rows of a matrix product:

$$h = \varphi(Wx + b), \quad W \in \mathbb{R}^{m \times p}, b \in \mathbb{R}^m.$$

The notation is compact but slightly misleading. φ is not an operation on the vector $Wx + b$; it is applied separately to each of its m entries, one entry per unit. The units do not interact within a layer.

m is the width. Layers compose: $h^{(1)} = \varphi(W_1 x + b_1)$, $h^{(2)} = \varphi(W_2 h^{(1)} + b_2)$, and so on. The number of layers is the depth. A layer that is neither the input nor the output is a hidden layer, and its entries are hidden units — hidden in the sense that nothing outside the network observes them.

The final layer is different: a single unit with no nonlinearity, a linear map from the last hidden layer to one number. So the network as a whole takes one runner’s features and returns one number, which is the score section 3 uses. A network of this shape is a multi-layer perceptron. Paper 5’s scorer has two hidden layers, of widths 256 and 128, and then the single output unit.

1.3 Universal approximation, stated honestly

A network with one hidden layer of sufficient width can approximate any continuous function on a compact set to arbitrary accuracy. This is a statement about existence: for a given target function and tolerance, such weights exist.

It does not say how wide the layer must be, and the width required can grow very quickly with the dimension of x . It does not say that gradient descent will find those weights; the optimisation problem is non-convex and nothing guarantees the global optimum is reached. And it says nothing at all about generalisation — the theorem concerns approximating a known function, not recovering one from a finite, noisy sample. It is a reason not to worry that the model class is too small. It is not a reason to expect a good fit.

2 Training

2.1 The loss and gradient descent

Fitting means choosing θ — all the weights and biases, in every layer, collected into one vector — to minimise a scalar loss $L(\theta)$ computed from the data. The basic method is gradient descent: repeatedly step against the gradient,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta),$$

with step size η , the learning rate. Too large and the iterates oscillate or diverge; too small and progress is slow.

2.2 Backpropagation

The gradient is obtained by the chain rule. Write the network as a composition $L = \ell \circ f_n \circ \dots \circ f_1$. The chain rule gives the derivative of L with respect to the parameters of layer i as a product of Jacobians of the layers above it. Evaluating that product from the loss end downward — reverse mode — lets every layer reuse the partial product already computed by the layers above, so the whole gradient costs a small constant multiple of one forward evaluation. That is backpropagation: not a separate idea from the chain rule, but the efficient order in which to apply it.

2.3 Optimisers, epochs, minibatches

Plain gradient descent uses the whole dataset for each step. Stochastic gradient descent uses a random subset — a minibatch — giving a noisier gradient at a fraction of the cost. One pass over the training data is an epoch.

Adam is the optimiser paper 5 uses. It keeps running averages of the gradient and of its elementwise square and scales each parameter's step by the ratio of the two, and it has its own learning rate, which remains the most important setting.

Dropout, used during training only, sets a random fraction of a layer's hidden units to zero at each step, which discourages the network from relying on any single unit. The fraction is a setting to be chosen, and in paper 5's grid it was the one that mattered most. Weight decay adds $\lambda\|\theta\|^2$ to the loss, shrinking parameters toward zero; paper 5 holds it fixed at 10^{-5} throughout rather than varying it.

2.4 Overfitting, and why a validation slice exists

A network with many parameters can fit the training data closely and still predict badly. The symptom is a training loss that keeps falling while the loss on data not used for fitting rises.

This creates a problem for every choice not made by gradient descent — width, depth, learning rate, dropout rate, how many epochs to train. Those cannot be selected on the training loss, which prefers more capacity and more epochs without limit. They also cannot be selected on the data reserved for the final estimate of performance: a quantity chosen to look good on a particular set is no longer an unbiased estimate of performance on that set.

The resolution is a third partition. The data is split into a fitting set, a validation set used to make those choices and to decide when to stop, and a test set that is scored once at the end and is otherwise not consulted. Paper 5 carves its validation slice out of the training period by date, so that no information from later races reaches an earlier fit.

3 The Plackett–Luce likelihood under a neural scorer

Before the algebra, the shape of the arrangement.

The network takes one runner's features and returns one number. It knows nothing about races, field sizes, or the other runners. It is a function from a feature vector to a score, applied once per horse.

Those numbers are then collected by race, and the likelihood is computed from them outside the network. The loss is low when the horses that finished first, second and third had high scores relative to the rest of their field.

To improve the model, ask of each runner: if its score had been slightly higher, would the loss have gone up or down, and by how much? That question has an exact answer, because the loss is an explicit formula in the scores. The answer comes out as the predicted probability minus what happened — positive where the model gave a horse more chance than the result justified, negative where it gave less.

That one number per runner is the entire message from the likelihood back to the network. It says which scores were too high, which were too low, and by how much.

The network then works backwards. It produced that runner's score from that runner's features through its weights, so it can compute how each weight would have to change to move the score in the direction wanted. That is the chain rule applied backwards, as in section 2.2.

The loss and the network are joined only through the scores. The likelihood is a function of the scores; the scores are functions of the weights. Composing the two makes the loss a function of the weights, which is what gradient descent needs.

This is also why the same likelihood serves papers 2b, 3 and 5 unchanged. It hands back one number per runner whatever produced the score — a linear predictor, a tree ensemble, or a network.

3.1 The likelihood

Papers 2b and 3 derive this likelihood; it is unchanged here, and the only difference is what computes the score.

Consider race r with J_r runners. Each runner j has features x_{rj} and receives a score

$$z_{rj} = f_{\theta}(x_{rj}) \in \mathbb{R},$$

where f_{θ} is any function with parameters θ — a linear map, a tree ensemble, or a network. In paper 5 it is the network of section 1, applied to one runner's features: one runner in, one number out. The Plackett–Luce model says that the horse finishing first is drawn from the field with probability proportional to $\exp(z)$; that the second is drawn from the remainder by the same rule; and so on. Truncating at depth k , and writing (s) for the horse that finished s -th and $R_{r,s}$ for the set still unplaced at stage s ,

$$\Pr(\text{observed top-}k \text{ order in race } r) = \prod_{s=1}^{S_r} \frac{\exp(z_{r,(s)})}{\sum_{j \in R_{r,s}} \exp(z_{rj})}, \quad S_r = \min(k, J_r - 1).$$

The loss is the negative log-likelihood, summed over races:

$$L(\theta) = \sum_r \ell_r, \quad \ell_r = - \sum_{s=1}^{S_r} \left[z_{r,(s)} - \log \sum_{j \in R_{r,s}} \exp(z_{rj}) \right].$$

At $k = 1$ this is the conditional logit: only the winner enters, and each race contributes one softmax term. Paper 5 uses $k = 3$, as papers 2b and 3 do.

3.2 What the gradient hands to the network

Differentiating ℓ_r with respect to a single score gives, for runner j ,

$$\frac{\partial \ell_r}{\partial z_{rj}} = \sum_{s=1}^{S_r} (p_{rj}^{(s)} - \mathbb{1}\{j = (s)\}) , \quad p_{rj}^{(s)} = \frac{\exp(z_{rj})}{\sum_{i \in R_{r,s}} \exp(z_{ri})}$$

for $j \in R_{r,s}$, and $p_{rj}^{(s)} = 0$ once j has been placed and removed. The quantity is “predicted probability minus what happened”, accumulated over the stages the runner survives to.

This is the entire interface between the likelihood and the model. What the optimiser needs is not this derivative but the gradient with respect to the parameters, and the chain rule writes that as a product accumulated over runner-rows:

$$\frac{\partial L}{\partial \theta} = \sum_r \sum_j \frac{\partial \ell_r}{\partial z_{rj}} \cdot \frac{\partial z_{rj}}{\partial \theta}.$$

The likelihood supplies the first factor and knows nothing about θ . The network supplies the second — how its output moves when its weights move — and knows nothing about the likelihood. Backpropagation multiplies them and accumulates over rows.

That product is also the precise sense in which the same objective serves paper 2b’s linear predictor, paper 3’s tree ensemble and paper 5’s network: the first factor is identical in all three, and only the second changes with the model class.

3.3 Race as the grouping unit

Two structural facts follow from the form of the likelihood.

Only within-race differences matter. Adding a constant c_r to every score in race r leaves every ratio unchanged: $\exp(z + c_r) / \sum \exp(z + c_r) = \exp(z) / \sum \exp(z)$. The scorer is therefore identified only up to a per-race constant.

Runners are coupled only through the denominator. The score z_{rj} depends on x_{rj} alone. Runners in the same race interact only through the normalising sum. Runners in different races do not interact at all.

3.4 Shared weights and variable field sizes

The same f_θ is applied to every runner, so a field of six and a field of sixteen need no provision in the network: a function from one runner’s features to one number does not depend on how many times it is applied.

4 What an MLP can represent that a linear score cannot

This section is specific to the likelihood above. The interesting differences are not the textbook ones about function classes in general; they follow from scoring being race-conditional.

4.1 The linear case

With $f_{\theta}(x) = \beta^{\top} x$, the comparison between two runners in the same race is

$$z_j - z_i = \beta^{\top} (x_j - x_i).$$

The model's opinion about any pair is a fixed linear functional of their feature difference. It is the same functional everywhere in feature space and for every race.

4.2 Interactions must be named

A linear model can only use a combination of two features if you build that combination as a column yourself — draw multiplied by course, say. Papers 1 and 2 did exactly that, with an explicit course-by-draw interaction. A network does not need you to.

The reason is what a single unit computes. With $\varphi(u) = \max(0, u)$, the unit is switched off wherever $w^{\top} x + b$ is negative and linear wherever it is positive. The boundary between the two is a line — a plane if there are three features, and its equivalent in more — and where that boundary falls is set by the weights, so the fitting chooses it rather than the analyst. Stack such units and the result is a function built from flat pieces, a different piece on each side of every boundary, so how much weight the network puts on one feature can depend on the values of the others.

This is not a guarantee that the network finds the right interaction, only that the model class contains it without it having been named.

4.3 Thresholds and saturation

In a linear model an extra day off changes the score by the same amount whether the horse ran last week or three years ago. That is probably wrong: a layoff matters up to a point and then stops mattering. To express it in a linear model you must decide the shape in advance, by taking a logarithm or by binning the feature.

A network works the shape out. A unit that switches off above a threshold gives a slope that changes at a point chosen by the fitting rather than by the analyst.

4.4 Race-level features

The going, the class, the course and the field size are the same for every horse in a race. A linear model adds the same amount to every score in that race, so it cancels and the feature does nothing at all. A network can use it to change how it reads the other features — weighting the draw more heavily at one course than at another — without adding a constant.

In symbols: write u_r for a feature constant across the runners of race r . A linear score contributes $\beta_u u_r$ to every runner in that race, and by section 3.3 a per-race constant cancels from every ratio in the likelihood. The coefficient is not identified, and including the column adds no capacity. The only way a linear model can use race-level information is through an explicit interaction with a runner-level feature, such as $u_r \times \text{draw}_{rj}$, which does vary within the race.

This matters for reading paper 5’s rung 1, where a network and a linear score were given an identical feature set including a going term and four course indicators — five columns that are constant within a race. On the linear side those five columns can do nothing whatever; on the network side they can modulate the remaining terms. Identical inputs, in this likelihood, do not mean identical usable inputs.

4.5 What the network does not buy

The network still sees one horse at a time, so it cannot tell whether the rest of the field is strong or weak. That needs a different kind of model: a function of the whole race rather than of one runner.

And extra flexibility applied to features that carry no signal fits noise. That is what the validation slice of section 2.4 exists to detect.

5 Embeddings

5.1 The construction

A categorical variable with V levels — a trainer identifier, say — cannot be fed to a network as an integer, since the arithmetic would imply an ordering that does not exist. The standard treatments are one-hot encoding, which represents level v by a basis vector $e_v \in \mathbb{R}^V$, and an embedding, which represents it by a learned vector.

An embedding is a matrix E with V rows and d columns: one row per level, each row a list of d numbers. The representation of level v is the v -th row of E . The product $e_v^\top E$ picks out exactly that row, so the lookup is a row lookup and the matrix product is the formal way of writing it; in code it is an array index.

The rows are parameters and are learned by the same gradient descent as every other weight. Only the row for level v receives gradient from a row of data carrying that level.

5.2 Why an embedding can carry more than a rate

A hand-built alternative is a summary statistic — for a trainer, the proportion of prior runners that won. That is one number per level, fixed by a formula chosen in advance.

An embedding is d numbers per level, and their meaning is determined by the objective. It can in principle encode that a trainer does well with first-time-out sprinters and badly over longer distances, because the embedding vector is concatenated with the other features before the nonlinearity, and the network can therefore make its use of the other features depend on which trainer this is. A single rate cannot express anything conditional; it enters as one more additive term.

5.3 The sample-size problem

The cost is parameters. The table holds $V \times d$ of them, and the row for level v is estimated only from rows carrying level v . Entity counts in this kind of data are heavily skewed: a few trainers with thousands of runners, a long tail with a handful each. A level appearing three times has a d -vector fitted from three observations.

The standard remedy is a frequency threshold: levels with at least $n_{\min}$ rows in the fitting data get their own row, and everything below is pooled into one shared bucket. This bounds the parameter count and prevents the rarest levels from carrying essentially untrained vectors, at the cost that all of them share one representation.

The vocabulary — which levels get their own row — must be fixed on the fitting data alone. Deriving it from the data the model is evaluated on would let that data determine the model’s structure.

5.4 The information-set point

There is a second difference between the two constructions, separate from what each can represent.

An embedding is not time-varying. A trainer’s row is one fixed vector, learned once from the fitting data and unchanged thereafter. A strike rate is recomputed after every race that trainer has, so it tracks a trainer improving or declining. The embedding cannot.

An embedding can also only learn about people who appear in the fitting data. A strike rate can be calculated for anyone with a record, including trainers who only appear later, and it uses their whole history rather than only the races the model was fitted on.

Under a chronological split the later period brings in trainers and jockeys the earlier one never saw. Rung 2 therefore compared two different amounts of information as much as two ways of representing the same information.

6 Sequence encoders

6.1 The problem

The input is now a variable-length sequence $x_1, \dots, x_T$, one vector per prior run of a horse. Each $x_t \in \mathbb{R}^c$, where c is the number of values describing a single run — eight in paper 5: where the horse finished, how far back it was beaten, the going, and so on. The output must be a fixed-length vector, because the scorer of section 3 takes a fixed-length input.

Averaging or summing the sequence produces a fixed-length vector but discards order: it gives the same answer for a horse improving through its last five runs and one declining through them.

6.2 Recurrence

A recurrent network maintains a running state and updates it once per element:

$$h_t = \varphi(Wx_t + Uh_{t-1} + b), \quad h_0 = 0.$$

The final state h_T is the summary. Order matters because h_{t-1} enters the update for h_t .

The difficulty is training. The gradient of the loss with respect to early elements passes through T successive multiplications by U and by derivatives of φ , so it tends to shrink or grow geometrically in T . Long-range dependence is then hard to learn.

6.3 The gated recurrent unit

The GRU addresses this with two gates. All quantities are vectors of the hidden width H , $\odot$ is elementwise multiplication, and σ is the logistic function, so each gate is a vector of numbers in $(0, 1)$:

$$\begin{aligned} z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{update gate} \\ r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{reset gate} \\ \tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) && \text{candidate state} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{new state} \end{aligned}$$

The last line is the substance. The new state is an interpolation between the previous state and the candidate, controlled elementwise by z_t . Where z_t is near zero the corresponding coordinate of the state is carried forward almost unchanged, which gives the gradient a near-identity path across that step; where z_t is near one the coordinate is replaced. The network learns when to hold and when to overwrite, per coordinate, so different hidden dimensions can operate over different spans of the sequence.

The reset gate controls how much of the previous state is visible when the candidate is formed. With r_t near zero the candidate depends on x_t alone, which lets a coordinate discard accumulated history when the current element is what matters.

Order changes the output, and not only in principle: h_T is a function of the sequence, and exchanging two elements changes it in general. This is the property that distinguishes a recurrent summary from an average.

The weights $W_z, U_z, W_r, U_r, W_h, U_h$ and their biases are learned from the same loss as the rest of the model, so what the encoder summarises is shaped by the prediction it serves; a tree cannot be trained that way, which is why paper 5 fits the encoder and the scorer together.